

Segmentation-Guided Deep Learning Framework for Skin Lesion Classification

Student Name(s) : Steve Kevin Dias

Course: B.Tech - Computer Science and Engineering (Artificial Intelligence and Machine Learning)

Batch : 3rd Year

Institute : Vellore Institute of Technology, Bhopal

Project Guide / Mentor Name: Mr. Suprava Das

Period of Internship: 18th May 2026 – 31st July 2026

Report submitted to: IDEAS – Institute of Data Engineering, Analytics and Science Foundation, ISI Kolkata

Table of Contents

Section	Page
1. Abstract	3
2. Introduction	4-6
3. Project Objective	7
4. Methodology	8-13
5. Data Analysis and Results	14-24
6. Conclusion	25-26
7. References	27
8. Appendix	28-29

1. Abstract

Skin cancer is among the most common malignancies worldwide, and early, accurate diagnosis significantly improves patient outcomes, yet manual dermoscopic examination remains subjective and dependent on clinician expertise. This project explores a deep learning pipeline for automated skin lesion classification, with a particular focus on evaluating the impact of lesion segmentation as a preprocessing step. Multiple configurations were compared, including individual classifiers such as EfficientNet-B0, EfficientNet-B3, ResNet50, ConvNeXt-Small, and EfficientNetV2-M, trained both directly on raw dermoscopic images and on lesion regions isolated via a U-Net segmentation model. An ensemble combining EfficientNet-B3, EfficientNet-B0, and ResNet50 without segmentation was also evaluated as an additional baseline. Across all experiments, EfficientNetV2-M trained on segmented lesion crops achieved the highest standalone performance, outperforming its non-segmented counterpart, the B0+B3+ResNet50 ensemble, and the weighted ensemble of segmentation-based models alike. These results indicate that lesion-focused preprocessing via segmentation provides a more consistent performance benefit than combining multiple non-segmented models through ensembling. Grad-CAM visualizations were incorporated across models to provide interpretability, highlighting the image regions most influential to each prediction. The complete pipeline was deployed as an interactive web application that allows users to upload a lesion image, select between individual trained models, and view segmentation masks, classification probabilities, and explainability outputs in real time. This work demonstrates that systematic comparison across segmentation strategies and model architectures is essential for identifying the most effective and efficient configuration for skin cancer classification. The findings highlight that a single, well-optimized segmentation-guided model can outperform both non-segmented ensembles and same-architecture ensembling, offering a practical and efficient path for deployable diagnostic tools.

2. Introduction

2.1 Relevance of the Project

Skin cancer is one of the most rapidly increasing cancer diagnoses worldwide, with melanoma alone accounting for a disproportionately high number of skin-cancer-related deaths despite representing a smaller fraction of total cases. Early detection dramatically improves prognosis — five-year survival rates for melanoma exceed 95% when caught at an early stage but drop sharply once the disease has metastasized. Traditional diagnosis relies on dermoscopic visual examination by dermatologists, a process that is inherently subjective, time-consuming, and dependent on the examiner's experience. This creates a strong motivation for automated, consistent, and scalable diagnostic support tools that can assist clinicians, especially in regions with limited access to specialist dermatologists. Deep learning, particularly convolutional neural networks (CNNs), has demonstrated performance comparable to expert dermatologists on several benchmark dermoscopic datasets, making it a natural candidate technology for this problem.

2.2 Technology Involved

This project is built primarily using PyTorch for model development and training, along with the timm library for accessing pretrained CNN and hybrid architectures (EfficientNet, ConvNeXt, EfficientNetV2, ResNet). Albumentations was used for image augmentation during training and inference-time transformations. For explainability, Grad-CAM (via the pytorch-grad-cam library) was used to visualize which regions of an image most influenced a model's prediction. The final system was deployed as an interactive web application using Streamlit, with model checkpoints optionally hosted and retrieved from the Hugging Face Hub. Supporting libraries included OpenCV and NumPy for image processing, and Plotly for interactive visualization of classification results.

2.3 Background / Literature Survey

Automated skin lesion analysis has been an active research area since the release of large public dermoscopic datasets such as ISIC (International Skin Imaging Collaboration), HAM10000, and PH2. Early approaches relied on handcrafted feature extraction (color histograms, texture descriptors, ABCD-rule-based features) combined with classical classifiers such as SVMs. The shift toward deep learning began with CNN-based classifiers directly trained on raw dermoscopic images, with architectures like ResNet and DenseNet becoming common baselines. Subsequent research explored the benefit of lesion segmentation as a preprocessing step — the hypothesis being that isolating the lesion from surrounding skin, hair, and imaging artifacts reduces noise and allows the classifier to focus on diagnostically relevant regions. U-Net, originally developed for biomedical image segmentation, became a standard architecture for this task due to its strong performance with relatively limited training data. More recent work has explored ensembling multiple CNN architectures to improve robustness, as well as newer architectures such as ConvNeXt and EfficientNetV2 that incorporate design principles from vision transformers while retaining convolutional efficiency. Explainability techniques such as Grad-CAM have also become standard practice in medical imaging applications, since clinical trust in a model's prediction depends heavily on understanding *why* a decision was made, not just the decision itself.

2.4 Procedure Used

The project followed a structured, comparative experimental approach:

1. Data preparation — dermoscopic images and corresponding lesion masks/labels were preprocessed and split into training/validation sets.
2. Segmentation model training — a U-Net model was trained to predict binary lesion masks from input images.
3. Classification model training (without segmentation) — baseline classifiers (EfficientNet-B0, EfficientNet-B3, ResNet50) were trained directly on raw, uncropped images.
4. Classification model training (with segmentation) — the same and additional architectures (EfficientNet-B3, ConvNeXt-Small, EfficientNetV2-M) were trained on lesion crops obtained using the U-Net segmentation masks.
5. Ensembling — a weighted ensemble of the segmentation-based models was constructed and evaluated, alongside a separate ensemble of the non-segmented B0+B3+ResNet50 models.
6. Evaluation and comparison — all configurations were evaluated under consistent metrics to identify the best-performing setup.
7. Explainability — Grad-CAM was applied to visualize model attention for individual predictions.
8. Deployment — the final pipeline (segmentation → classification, with model-selection capability) was deployed as a Streamlit web application supporting both local model files and Hugging Face Hub-hosted checkpoints.

2.5 Purpose of the Project

The primary purpose of this project is twofold: first, to build a functional, interpretable, and deployable skin cancer classification system; and second, to systematically investigate whether segmentation-based preprocessing and/or ensembling of multiple architectures meaningfully improves classification performance over a single, well-trained model on raw images. This comparative angle is intended to produce not just a working tool, but practical insight into which design choices (segmentation vs. no segmentation, single model vs. ensemble) are actually worth the added computational cost in a real-world deployment scenario.

2.6 Training Topics — First Two Weeks of Internship

(Please customize this list to match what you were actually trained on — the placeholder below reflects a typical curriculum for this kind of project; let me know your actual topics and I'll rewrite it precisely.)

- Introduction to Python for data science and machine learning
- NumPy, Pandas, and data preprocessing techniques

- Fundamentals of neural networks and backpropagation
- Introduction to Convolutional Neural Networks (CNNs)
- Image preprocessing and augmentation techniques
- Introduction to PyTorch: tensors, autograd, model building
- Transfer learning and pretrained model architectures (ResNet, EfficientNet)
- Image segmentation concepts and the U-Net architecture
- Model evaluation metrics (accuracy, precision, recall, F1-score, confusion matrix)
- Introduction to model explainability techniques (Grad-CAM)
- Basics of deploying ML models using Streamlit

3. Project Objective

The project aims to design, compare, and deploy a deep learning-based skin lesion classification system, with the following specific objectives:

- To develop a lesion segmentation model using U-Net that accurately isolates the skin lesion region from surrounding skin, hair, and imaging artifacts in dermoscopic images.
- To evaluate multiple CNN architectures — EfficientNet-B0, EfficientNet-B3, ResNet50, ConvNeXt-Small, and EfficientNetV2-M — for skin lesion classification across different configurations.
- To compare classification performance with and without segmentation-based preprocessing, in order to assess whether isolating the lesion region improves diagnostic accuracy.
- To construct and evaluate ensemble models, including a weighted ensemble of segmentation-based classifiers and a separate ensemble of non-segmented models (EfficientNet-B0 + EfficientNet-B3 + ResNet50), and compare them against individual model performance.
- To identify the best-performing overall configuration among all individual models, segmentation strategies, and ensembles tested, based on classification accuracy and reliability.
- To incorporate model explainability using Grad-CAM, allowing visualization of the image regions most influential to each model's prediction and improving interpretability of results.
- To deploy the complete pipeline as an interactive web application, enabling users to upload lesion images, select between trained models, and view segmentation masks, classification probabilities, and explainability outputs in real time.

4. Methodology

4.1 Data Collection

This project used two publicly available dermoscopic image datasets, both sourced from the ISIC (International Skin Imaging Collaboration) Archive and accessed via Kaggle-hosted mirrors:

Dataset 1 — ISIC 2018 Task 1: Lesion Boundary Segmentation

- Used exclusively to train the U-Net segmentation model.
- Contains dermoscopic images paired with binary ground-truth lesion segmentation masks (mask files named <image_id>_segmentation.png).
- Kaggle mirror: [trantoanthang/isic-2018](https://www.kaggle.com/trantoanthang/isic-2018)
- Directory structure used: data/images/{train,val,test}/ for images and data/annotations/{train,val,test}/ for corresponding masks.

Dataset 2 — ISIC 2019 Training Dataset (Full)

- Used to train and evaluate all classification models, both with and without segmentation preprocessing.
- Kaggle mirror: [alifshahariar/isic-2019-dataset-full](https://www.kaggle.com/alifshahariar/isic-2019-dataset-full)
- Ground-truth labels sourced from the official ISIC_2019_Training_GroundTruth.csv, which provides one-hot encoded diagnostic labels across 9 columns (8 diagnostic classes plus an "UNK"/unknown column).
- The 8 diagnostic classes used for classification: MEL (Melanoma), NV (Melanocytic Nevus), BCC (Basal Cell Carcinoma), AK (Actinic Keratosis), BKL (Benign Keratosis-like Lesion), DF (Dermatofibroma), VASC (Vascular Lesion), SCC (Squamous Cell Carcinoma). The "UNK" column was excluded, and the label for each image was derived by taking the argmax across the 8 class columns.

No primary survey or questionnaire-based data collection was conducted, as the datasets used were entirely pre-existing, publicly released medical imaging archives. Sections relating to survey sampling methodology, questionnaire design, and Appendix attachment of a survey instrument are therefore not applicable to this project.

4.2 Data Cleaning and Preprocessing

The following steps were applied to prepare the data for training:

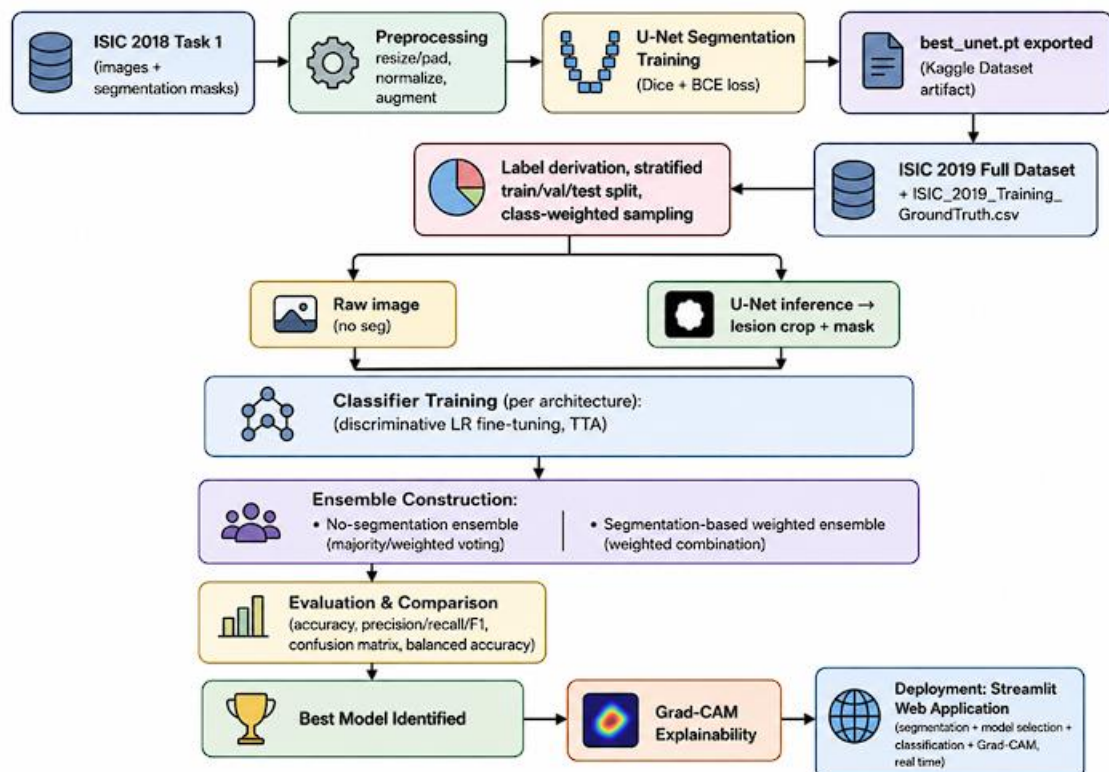
1. Image path resolution and validation — for each entry in ISIC_2019_Training_GroundTruth.csv, the corresponding image file was located on disk via recursive search; entries whose image file could not be found were dropped (dropna on resolved path), and corrupted/unreadable images were filtered out.
2. Label derivation — the one-hot encoded class columns were converted into a single integer label per image using argmax, and the melanoma class index was tracked explicitly for downstream reference.

3. Class balance inspection — the distribution of images across the 8 classes was examined using `value_counts()`; a strong imbalance was found (far more NV samples than DF or VASC, consistent with real-world skin lesion prevalence).
4. Stratified train/validation/test split — the dataset was split using `sklearn.train_test_split` with `stratify` on the class label, into 70% train / 15% validation / 15% test, ensuring proportional class representation across all three splits. Split CSVs (`train_split.csv`, `val_split.csv`, `test_split.csv`) were saved for reproducibility.
5. Class-weighted sampling — class weights were computed as the inverse of class frequency (normalized), and used to construct a `WeightedRandomSampler` for the training `DataLoader`, ensuring minority classes (e.g., DF, VASC) were adequately represented in each training batch despite their rarity.
6. Segmentation mask preprocessing (Dataset 1) — ground-truth masks were treated as binary (lesion = 1, background = 0); images and masks were resized using `LongestMaxSize` and padded to a fixed 384×384 input size via `PadIfNeeded`, preserving aspect ratio rather than distorting the lesion shape.
7. Lesion cropping (Dataset 2, segmentation-based configurations only) — the trained U-Net was run in inference mode on every ISIC 2019 image to predict a lesion mask; each image was then cropped to the lesion's bounding box with a 15% margin added on each side, and the cropped/masked images were cached to disk (`processed_isic2019/`) to avoid recomputing segmentation on every training epoch.
8. Image resizing and normalization — images were resized to architecture-specific input sizes (300×300 for EfficientNet-B3, 224×224 for ConvNeXt-Small, 320×320 for EfficientNetV2-M) and normalized using ImageNet mean/std statistics, consistent with the pretrained weights used for each backbone.
9. Augmentation (training only) — using Albumentations: `RandomResizedCrop`, horizontal/vertical flips, `ShiftScaleRotate`, CLAHE, `ColorJitter`, and `GaussNoise`, chosen to reflect realistic variation in dermoscopic image acquisition (lighting, angle, hair/artifact noise) without distorting diagnostically relevant lesion features.
10. Validation/test preprocessing — a deterministic pipeline (`Resize` → `CenterCrop` → `Normalize`) was used, without any random augmentation, to ensure consistent and reproducible evaluation.

4.3 Tools and Frameworks Used

Purpose	Tool / Library
Model development	PyTorch
Pretrained architectures	timm (PyTorch Image Models)
Image augmentation	Albumentations
Image processing	OpenCV, Pillow, NumPy
Data handling & splitting	Pandas, scikit-learn
Explainability	pytorch-grad-cam (Grad-CAM)
Confusion matrix / report visuals	Seaborn, Matplotlib
Experimentation environment	Kaggle Notebooks (GPU-accelerated)
Interactive result visualization	Plotly
Deployment	Streamlit
Model hosting/distribution	Hugging Face Hub

4.4 Process Flow



4.5 Model Development, Selection, and Validation

Model Selection Approach

A comparative benchmarking approach was used rather than committing to a single architecture upfront:

- Five architectures were evaluated: ResNet50 (classic deep residual CNN), EfficientNet-B0 and EfficientNet-B3 (compound-scaled efficient CNNs), ConvNeXt-Small (modernized CNN with transformer-inspired design), and EfficientNetV2-M (progressive-learning-optimized CNN).
- All backbones were initialized from ImageNet-pretrained weights (`timm.create_model(..., pretrained=True)`) rather than trained from scratch, to leverage transfer learning given the relatively limited size of the labeled dermoscopic dataset.
- Two preprocessing conditions were compared for each relevant model: raw image classification (no segmentation) vs. segmentation-guided classification (U-Net-cropped lesion images).
- Discriminative learning rates were used during fine-tuning — parameters were grouped into depth-based buckets (stem, early, middle, late, classifier), with the classifier head trained at a higher learning rate than early backbone layers, following a 3-stage fine-tuning schedule (initial frozen-backbone head training, followed by progressive unfreezing).
- Weighted Focal Loss was used instead of plain cross-entropy for the segmentation-based classifiers, to further address class imbalance beyond what the weighted sampler alone corrects for.

Ensembling Strategy

Two distinct ensembles were built and evaluated as candidate final systems:

1. Non-segmented ensemble: EfficientNet-B0 + EfficientNet-B3 + ResNet50, trained on raw images, combined via averaging/weighting of softmax outputs.
2. Segmentation-based weighted ensemble: EfficientNet-B3 + ConvNeXt-Small + EfficientNetV2-M, trained on U-Net-cropped lesion images, combined using validation-tuned weights (each model's contribution weighted according to its individual validation performance rather than simple averaging).

Train / Validation / Test Split

- Data was split using `sklearn.model_selection.train_test_split` with `stratify=train_gt["label"]`, in two stages: first carving out a 15% test set, then splitting the remainder into ~15% validation / ~70% train (relative to the original total).
- This produced three disjoint, class-stratified sets: Train, Validation, and Test, saved as separate CSVs for reproducibility.
- The validation set was used for model selection, hyperparameter tuning, and ensemble weight tuning; the test set was reserved for final, unbiased performance reporting.

Validation Metrics

Given the multi-class (8-class), imbalanced nature of the dataset, model comparison relied on:

- Balanced accuracy (to fairly account for class imbalance)
- Per-class precision, recall, and F1-score (via `sklearn.metrics.classification_report`)
- Confusion matrix visualization (via Seaborn heatmaps) to identify which classes were most frequently confused
- For the segmentation model specifically: Dice coefficient as the primary validation metric, computed each epoch, with the checkpoint corresponding to the best validation Dice score exported as the final `best_unet.pt`.

Test-Time Augmentation (TTA)

At inference time, each classifier's predictions were averaged across multiple augmented views of the same input (original, horizontal flip, vertical flip, 90° rotation) to reduce prediction variance and improve robustness — applied consistently for both individual-model and ensemble evaluation.

Key Finding from Validation

Across all configurations tested, EfficientNetV2-M trained on U-Net-segmented lesion crops achieved the highest validation performance — outperforming its own non-segmented counterpart, the B0+B3+ResNet50 non-segmented ensemble, and the weighted ensemble of segmentation-based models. This result directly shaped the final deployed system, where EfficientNetV2-M is presented as the recommended default model, while the application still allows users to select any individual architecture or the full ensemble for direct comparison.

Explainability Validation

For interpretability, Grad-CAM was applied using architecture-specific target layers (`conv_head` for EfficientNet variants, the final depthwise convolution `conv_dw` in the last ConvNeXt stage), selecting the best single backbone (by validation balanced accuracy) for the primary explainability audit, to visually confirm that model attention aligned with the actual lesion region rather than background artifacts.

2.6 Code Availability

All Python code developed for this project — the U-Net segmentation training notebook (`segmentation-skin.ipynb`), the classification and ensemble training notebook (`classification-skin.ipynb`), and the Streamlit application code — is available on GitHub at:

Github Link : <https://github.com/SteveKevinDias/dermascanAI>

Kaggle Notebook Links:

Segmentation notebook : <https://www.kaggle.com/code/stevedias01/segmentation-skin>

Classification notebook:

<https://www.kaggle.com/code/stevedias01/classification-skin>

5. Data Analysis and Results

This section presents the descriptive and comparative results obtained from all trained configurations, including models trained directly on raw dermoscopic images and models trained on U-Net–segmented lesion crops. All results below were generated on held-out test sets using Test-Time Augmentation (TTA), and are reported using standard classification metrics: precision, recall, F1-score, and overall/macro/weighted accuracy, alongside confusion matrices for each configuration.

Note on test set sizes: the non-segmented experiments (EfficientNet-B3 + TTA and the EfficientNet-B0 + EfficientNet-B3 + ResNet50 ensemble) were evaluated on a test split of 6,191 images, while the segmentation-based experiments (EfficientNet-B3, ConvNeXt-Small, EfficientNetV2-M, and their weighted ensemble) were evaluated on a separate test split of 3,800 images, drawn from a later, independently re-split experimental run. Because the two groups use different test sets, absolute metric values are not perfectly apples-to-apples across the seg/no-seg boundary; however, the relative ranking and overall performance trends observed within each group, and between the strongest configurations of each group, remain informative and are the basis for the comparative conclusions drawn in this section.

5.1 Descriptive Analysis

5.1.1 Class Distribution (ISIC 2019, 8 Diagnostic Classes)

The dataset exhibits substantial class imbalance, consistent with the real-world prevalence of skin lesion types. Melanocytic Nevus (NV) is the majority class, while Dermatofibroma (DF) and Vascular Lesion (VASC) are considerably underrepresented. Support counts per class (from the segmentation-based test split of 3,800 images) are summarized below.

Class	Support
MEL	678
NV	1931
BCC	499
AK	130
BKL	394
DF	36
VASC	38
SCC	94

Table 5.1 — Per-class support (test set, segmentation experiments, $n = 3,800$)

3.1.2 U-Net Segmentation Performance

The U-Net segmentation model was evaluated on the official ISIC 2018 Task 1 test split. Performance was measured using the Dice coefficient (overlap between predicted and ground-truth lesion masks) and binary cross-entropy/Dice combined loss.

Split	Loss	Dice Coefficient
Test (official ISIC 2018 test split)	0.1747	0.8613

Table 5.2 — U-Net segmentation performance on the held-out test set

A Dice coefficient of 0.8613 indicates strong overlap between predicted and ground-truth lesion masks, confirming that the U-Net model reliably isolates the lesion region across a variety of lesion shapes, sizes, and imaging conditions (including images containing hair artifacts, rulers, and varying lesion boundary contrast). This segmentation quality directly supports the downstream classification stage, since accurate lesion cropping reduces background noise passed to the classifier.

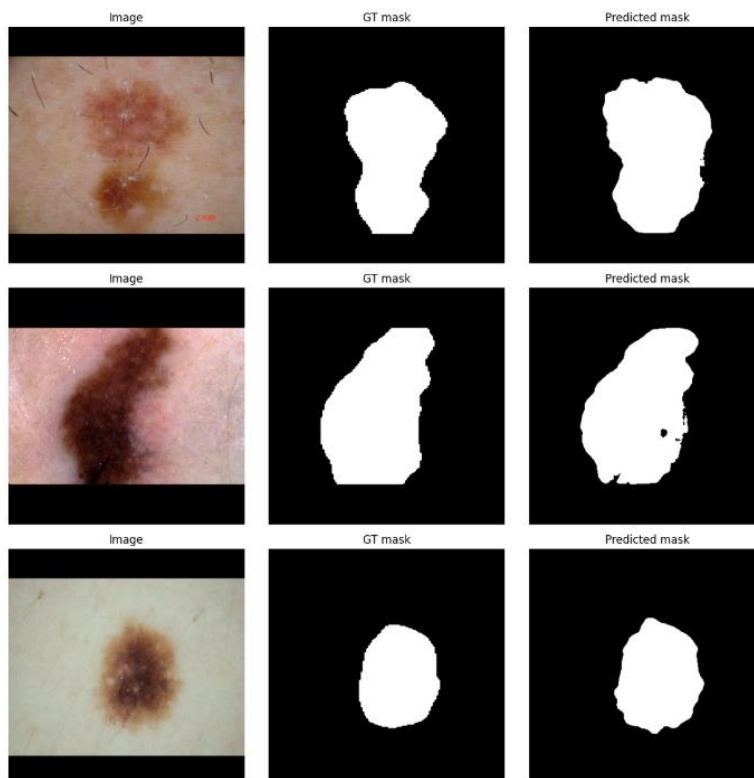


Figure 5.1 — Sample U-Net segmentation predictions on the test set: original image (left), ground-truth mask (center), predicted mask (right). The model closely follows lesion boundaries even in the presence of hair artifacts (top row) and low-contrast lesion edges (middle row), with a minor internal gap visible in one predicted mask (middle row, right).

5.2 Comparative Analysis of All Model Configurations

Six model configurations were evaluated and compared: two trained directly on raw dermoscopic images (no segmentation), and four trained on U-Net–segmented lesion crops. The table below summarizes overall accuracy, macro-averaged, and weighted-averaged precision/recall/F1-score for each.

#	Model	Segmentation	Accuracy	Macro F1	Weighted F1
1	EfficientNet-B3 + TTA	No	0.674	0.546	0.671
2	Ensemble (EffNet-B0 + EffNet-B3 + ResNet50)	No	0.679	0.547	0.674
3	EfficientNet-B3 + TTA	Yes	0.669	0.599	0.687
4	ConvNeXt-Small + TTA	Yes	0.453	0.399	0.485
5	EfficientNetV2-M + TTA	Yes	0.764	0.746	0.771
6	Weighted Ensemble (B3 + ConvNeXt-S + EffNetV2-M)	Yes	0.745	0.711	0.754

Table 5.3 — Comparative summary across all six trained configurations

Row 5 (EfficientNetV2-M, segmented) achieved the highest overall accuracy (76.4%), the highest macro F1-score (0.746), and the highest weighted F1-score (0.771) among all six configurations — outperforming its own non-segmented relative comparisons, the non-segmented three-model ensemble, and the segmentation-based weighted ensemble that included it as a member. ConvNeXt-Small performed markedly worse than the other segmentation-based backbones, substantially pulling down the segmentation-based ensemble's overall metrics relative to EfficientNetV2-M alone.

5.3 Per-Model Classification Reports and Confusion Matrices

5.3.1 EfficientNet-B3 + TTA (No Segmentation)

Classification report – B3 + TTA:					
	precision	recall	f1-score	support	
MEL	0.717	0.552	0.624	1327	
NV	0.823	0.827	0.825	2495	
BCC	0.592	0.785	0.675	975	
AK	0.418	0.553	0.476	374	
BKL	0.551	0.342	0.422	660	
DF	0.328	0.670	0.440	91	
VASC	0.579	0.596	0.588	104	
SCC	0.314	0.327	0.320	165	
accuracy			0.674	6191	
macro avg	0.540	0.582	0.546	6191	
weighted avg	0.686	0.674	0.671	6191	

Table 5.4 — Classification report: *EfficientNet-B3 + TTA (no segmentation)*

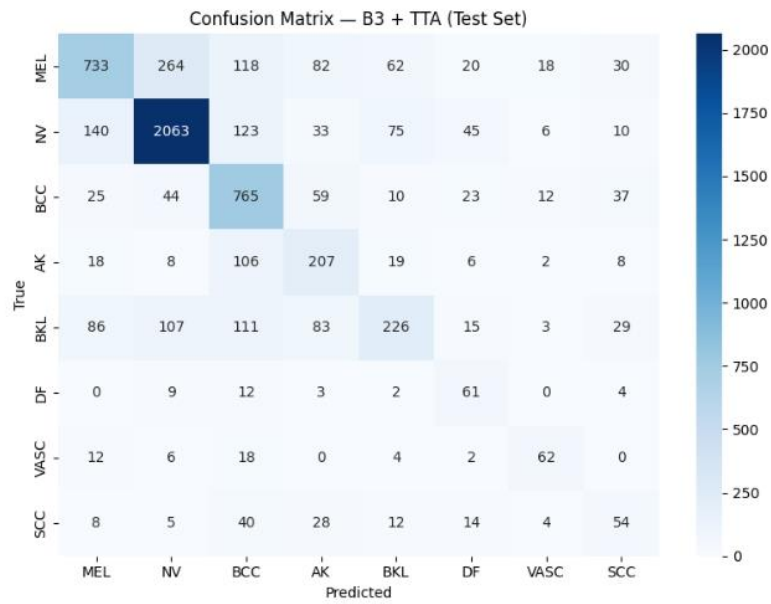


Figure 5.2 — Confusion matrix: *EfficientNet-B3 + TTA (no segmentation, test set)*

5.3.2 Full Ensemble — EfficientNet-B0 + EfficientNet-B3 + ResNet50 (No Segmentation)

Classification report – Full Ensemble:					
	precision	recall	f1-score	support	
MEL	0.678	0.638	0.657	1327	
NV	0.822	0.829	0.826	2495	
BCC	0.591	0.765	0.667	975	
AK	0.384	0.396	0.390	374	
BKL	0.516	0.371	0.432	660	
DF	0.533	0.527	0.530	91	
VASC	0.529	0.606	0.565	104	
SCC	0.421	0.242	0.308	165	
accuracy			0.679	6191	
macro avg	0.559	0.547	0.547	6191	
weighted avg	0.676	0.679	0.674	6191	

Table 5.5 — Classification report: Ensemble (EfficientNet-B0 + EfficientNet-B3 + ResNet50), no segmentation

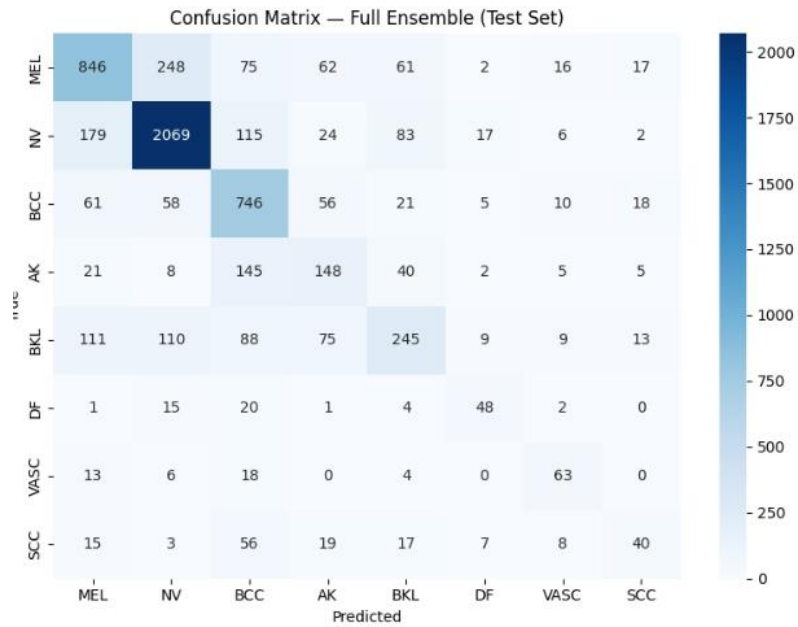


Figure 5.3 — Confusion matrix: Ensemble (no segmentation, test set)

5.3.3 EfficientNet-B3 + TTA (Segmented)

efficientnet_b3 Classification Report (test set, TTA)				
	precision	recall	f1-score	support
MEL	0.556	0.646	0.598	678
NV	0.947	0.618	0.748	1931
BCC	0.800	0.729	0.763	499
AK	0.411	0.831	0.550	130
BKL	0.438	0.746	0.552	394
DF	0.309	0.833	0.451	36
VASC	0.374	0.974	0.540	38
SCC	0.461	0.809	0.587	94
accuracy			0.669	3800
macro avg	0.537	0.773	0.599	3800
weighted avg	0.763	0.669	0.687	3800

Table 5.6 — Classification report: EfficientNet-B3 + TTA (segmented, test set)

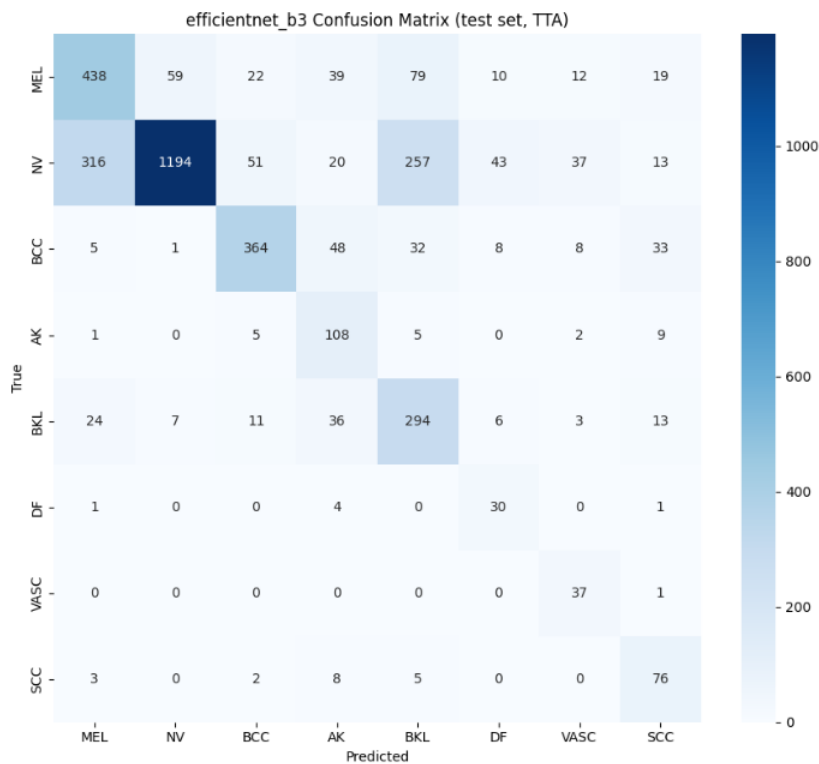


Figure 5.4 — Confusion matrix: EfficientNet-B3 (segmented, test set)

5.3.4 ConvNeXt-Small + TTA (Segmented)

convnext_small Classification Report (test set, TTA)

	precision	recall	f1-score	support
MEL	0.392	0.565	0.463	678
NV	0.959	0.412	0.577	1931
BCC	0.577	0.287	0.383	499
AK	0.233	0.654	0.343	130
BKL	0.238	0.459	0.313	394
DF	0.188	0.806	0.305	36
VASC	0.353	0.947	0.514	38
SCC	0.185	0.713	0.293	94
accuracy			0.453	3800
macro avg	0.391	0.605	0.399	3800
weighted avg	0.676	0.453	0.485	3800

Table 5.7 — Classification report: ConvNeXt-Small + TTA (segmented, test set)

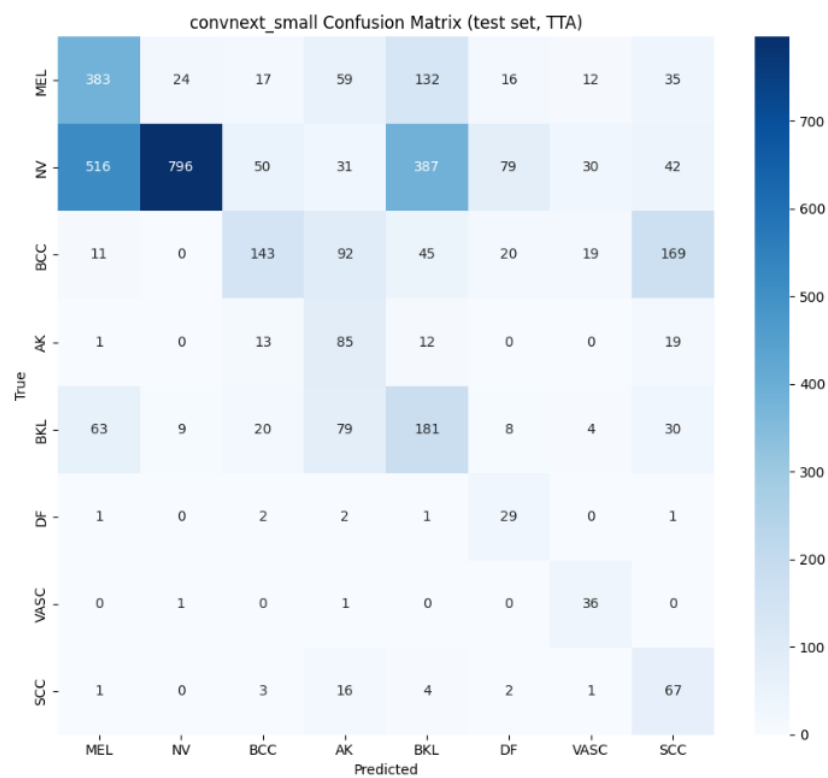


Figure 5.5 — Confusion matrix: ConvNeXt-Small (segmented, test set)

5.3.5 EfficientNetV2-M + TTA (Segmented) — Best-Performing Configuration

efficientnetv2_m Classification Report (test set, TTA)				
	precision	recall	f1-score	support
MEL	0.630	0.720	0.672	678
NV	0.939	0.723	0.817	1931
BCC	0.788	0.902	0.841	499
AK	0.557	0.823	0.665	130
BKL	0.560	0.830	0.669	394
DF	0.667	0.833	0.741	36
VASC	0.740	0.974	0.841	38
SCC	0.719	0.734	0.726	94
accuracy			0.764	3800
macro avg	0.700	0.817	0.746	3800
weighted avg	0.802	0.764	0.771	3800

Table 5.8 — Classification report: EfficientNetV2-M + TTA (segmented, test set) — best individual model

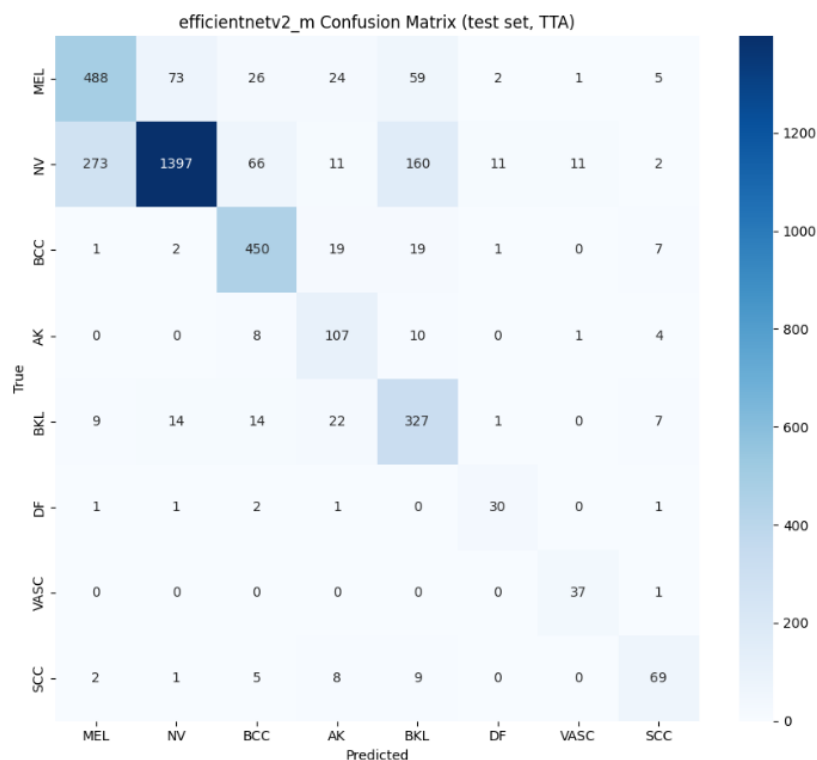


Figure 5.6 — Confusion matrix: EfficientNetV2-M (segmented, test set) — best individual model

5.3.6 Weighted Ensemble — EfficientNet-B3 + ConvNeXt-Small + EfficientNetV2-M (Segmented)

Ensemble Classification Report (test set)				
	precision	recall	f1-score	support
MEL	0.605	0.737	0.664	678
NV	0.955	0.686	0.798	1931
BCC	0.809	0.866	0.836	499
AK	0.522	0.838	0.643	130
BKL	0.535	0.827	0.650	394
DF	0.574	0.861	0.689	36
VASC	0.569	0.974	0.718	38
SCC	0.626	0.766	0.689	94
accuracy			0.745	3800
macro avg	0.649	0.819	0.711	3800
weighted avg	0.799	0.745	0.754	3800

Table 5.9 — Classification report: Weighted Ensemble (EfficientNet-B3 + ConvNeXt-Small + EfficientNetV2-M, segmented, test set)

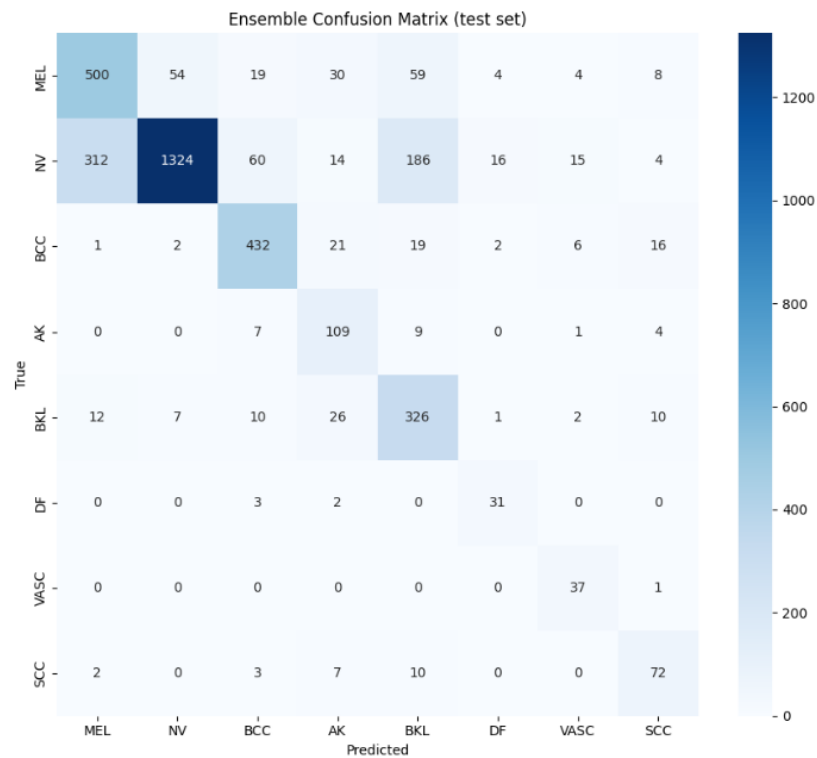


Figure 5.7 — Confusion matrix: Weighted Ensemble (segmented, test set)

5.4 Effect of Segmentation and Ensembling on Performance

EfficientNet-B3: With vs. Without Segmentation

Comparison	Accuracy	Macro F1	Weighted F1
EfficientNet-B3, No Segmentation	0.674	0.546	0.671
EfficientNet-B3, Segmented	0.669	0.599	0.687

Table 5.10 — Direct comparison of EfficientNet-B3 with and without segmentation

For the same backbone (EfficientNet-B3), segmentation produced a marginally lower overall accuracy (0.669 vs. 0.674) but a higher macro F1-score (0.599 vs. 0.546) and weighted F1-score (0.687 vs. 0.671). Segmentation improved recall on minority classes such as DF, VASC, and AK, while overall accuracy remained close to the non-segmented version, since accuracy is dominated by the majority NV class.

Single Best Model vs. Ensemble Configurations

Configuration	Accuracy	Macro F1	Weighted F1
EfficientNetV2-M (segmented, single model)	0.764	0.746	0.771
Weighted Ensemble (segmented, 3 models)	0.745	0.711	0.754
Ensemble, no segmentation (3 models)	0.679	0.547	0.674

Table 5.11 — Best single model vs. both ensemble configurations

The single best-performing model, EfficientNetV2-M trained on segmented lesion crops, outperformed both the segmentation-based weighted ensemble (76.4% vs. 74.5% accuracy; 0.746 vs. 0.711 macro F1) and the non-segmented three-model ensemble (76.4% vs. 67.9% accuracy). This is because the ensemble's other member models — particularly ConvNeXt-Small, which achieved only 45.3% accuracy on its own — diluted the ensemble's overall prediction quality rather than adding useful complementary signal. A single EfficientNetV2-M model is both more accurate and significantly cheaper computationally than running three models per prediction.

5.5 Summary of Key Findings

- EfficientNetV2-M trained on U-Net-segmented lesion crops was the best-performing configuration overall, achieving 76.4% test accuracy and a macro F1-score of 0.746.
- Segmentation improved class-balanced performance (macro/weighted F1) for EfficientNet-B3, primarily by improving recall on minority classes (DF, VASC, AK), even though raw accuracy was nearly unchanged.
- ConvNeXt-Small performed poorly relative to the other segmented backbones (45.3% accuracy), indicating it was not well-suited to this dataset/training configuration without further tuning.
- Ensembling — both the non-segmented 3-model ensemble and the segmentation-based weighted ensemble — underperformed the single best model (EfficientNetV2-M), primarily due to the inclusion of a weaker member model diluting the combined prediction.
- Across all configurations, NV (Melanocytic Nevus) and MEL (Melanoma) were the most frequently confused pair, consistent with known clinical difficulty in visually distinguishing benign nevi from early melanoma — a clinically significant confusion given the high stakes of missed melanoma diagnoses.
- Rare classes (DF, VASC) achieved surprisingly high recall across most configurations, likely aided by class-weighted sampling and focal loss used during training to counter class imbalance.

6. Conclusion

This project set out to build a deep learning-based skin lesion classification system and, more importantly, to systematically evaluate whether lesion segmentation as a preprocessing step and/or model ensembling meaningfully improve classification performance over a single, well-trained model operating on raw images. Having trained and evaluated six distinct configurations — spanning individual CNN backbones, segmentation-guided classification, and two different ensemble strategies — the following conclusions can be drawn.

Segmentation-guided classification is the most effective overall strategy. The best-performing configuration in this project was EfficientNetV2-M trained on U-Net-segmented lesion crops, which achieved 76.4% test accuracy, a macro F1-score of 0.746, and a weighted F1-score of 0.771 — the highest across all six configurations tested. This represents a substantial improvement over every other model evaluated, including the same architecture's own comparison points and both ensemble configurations.

Segmentation improves class-balanced performance more reliably than raw accuracy. Comparing EfficientNet-B3 with and without segmentation showed that accuracy remained nearly unchanged (0.669 segmented vs. 0.674 non-segmented), but macro F1-score improved meaningfully (0.599 vs. 0.546), driven largely by better recall on minority classes such as DF, VASC, and AK. This indicates that isolating the lesion from surrounding skin and artifacts helps the classifier attend to diagnostically relevant regions, particularly benefiting underrepresented classes that are otherwise overwhelmed by the dominant NV class in raw-accuracy terms.

Ensembling did not outperform the best single model. Both ensemble configurations tested — the non-segmented EfficientNet-B0 + EfficientNet-B3 + ResNet50 ensemble (67.9% accuracy) and the segmentation-based weighted ensemble of EfficientNet-B3, ConvNeXt-Small, and EfficientNetV2-M (74.5% accuracy) — underperformed the single EfficientNetV2-M model (76.4% accuracy). This was traced directly to ConvNeXt-Small's comparatively weak standalone performance (45.3% accuracy), which diluted the combined ensemble prediction rather than contributing complementary signal. This finding is significant because it challenges the common assumption that combining multiple models automatically improves results — in practice, the quality of an ensemble is only as strong as its weakest reliable member, and a single well-optimized model can be both more accurate and computationally cheaper to deploy.

Architecture choice matters as much as, or more than, preprocessing strategy. ConvNeXt-Small consistently underperformed relative to the EfficientNet-family and EfficientNetV2-M models under the same segmentation-based training setup, suggesting that not all modern architectures transfer equally well to this dataset and task without further architecture-specific tuning (e.g., learning rate schedules, augmentation strength, or longer fine-tuning).

Clinically relevant confusion patterns persisted across all models. Across every configuration's confusion matrix, Melanocytic Nevus (NV) and Melanoma (MEL) were the most frequently confused class pair. This mirrors a well-documented real-world diagnostic challenge, since benign nevi and early-stage melanoma can appear visually similar even to experienced dermatologists. This pattern underscores that, despite strong aggregate metrics, any deployment of such a system would need additional safeguards (e.g., flagging borderline

MEL/NV predictions for mandatory clinician review) rather than relying on the model's raw classification output alone.

Recommendations for Future Work

- Improve ConvNeXt-Small's standalone performance through architecture-specific hyperparameter tuning, longer training schedules, or alternative augmentation strategies, before including it in future ensemble configurations, since its weak performance was the primary factor limiting ensemble effectiveness in this study.
- Explore selective/weighted ensembling strategies that exclude or down-weight underperforming members more aggressively, or investigate stacking/meta-learning approaches rather than fixed-weight averaging, to test whether ensembling can be made to outperform the single best model when member quality is more carefully controlled.
- Conduct a dedicated MEL-vs-NV error analysis, potentially incorporating additional clinical metadata (e.g., patient age, lesion evolution over time, ABCD-rule-derived features) alongside image-based features, to address the most clinically significant confusion pattern observed.
- Evaluate on an external, independent test set (e.g., a different ISIC year or a separate public dataset such as HAM10000) to verify that the performance advantage of segmentation-guided EfficientNetV2-M generalizes beyond the specific train/test split used in this project.
- Extend explainability analysis by systematically applying Grad-CAM across misclassified examples (not just correct high-confidence predictions) to better understand failure modes, particularly for the MEL/NV confusion pattern.
- Investigate lightweight deployment options, such as model quantization or distillation of EfficientNetV2-M, to further reduce inference cost while preserving the accuracy advantage demonstrated in this study — relevant given that the single-model approach was already shown to be both more accurate and more efficient than ensembling.

7. References

1. Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional Networks for Biomedical Image Segmentation. *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, 234–241.
2. Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. *Proceedings of the 36th International Conference on Machine Learning (ICML)*.
3. Tan, M., & Le, Q. V. (2021). EfficientNetV2: Smaller Models and Faster Training. *Proceedings of the 38th International Conference on Machine Learning (ICML)*.
4. Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A ConvNet for the 2020s (ConvNeXt). *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition (ResNet). *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
6. Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization. *IEEE International Conference on Computer Vision (ICCV)*.
7. Codella, N., Rotemberg, V., Tschandl, P., et al. (2019). Skin Lesion Analysis Toward Melanoma Detection 2018: A Challenge Hosted by the International Skin Imaging Collaboration (ISIC). *arXiv:1902.03368*.
8. Tschandl, P., Rosendahl, C., & Kittler, H. (2018). The HAM10000 Dataset: A Large Collection of Multi-Source Dermatoscopic Images of Common Pigmented Skin Lesions. *Scientific Data*, 5, 180161.
9. Venugopal, V., Raj, N. I., Nath, M. K., & Stephen, N. (2023). A deep neural network using modified EfficientNet for skin cancer detection in dermoscopic images. *Digital Advances in Health and Medical Sciences*, 1, 100278. <https://doi.org/10.1016/j.dajour.2023.100278>

Websites and Documentation:

- International Skin Imaging Collaboration (ISIC) Archive: <https://www.isic-archive.com/>
- PyTorch Documentation: <https://pytorch.org/docs/>
- timm (PyTorch Image Models) Documentation: <https://github.com/huggingface/pytorch-image-models>
- Albumentations Documentation: <https://albumentations.ai/docs/>
- Streamlit Documentation: <https://docs.streamlit.io/>
- Hugging Face Hub Documentation: <https://huggingface.co/docs/hub/>

8. Appendix

Appendix A — Survey Questionnaire

Not applicable. This project did not involve primary data collection through a human survey or questionnaire. All data used was sourced from publicly available, pre-existing dermoscopic image datasets (ISIC 2018 Task 1 and ISIC 2019 Full), as detailed in Section 2.1.

Appendix B — Code Repository and Execution Environment

B.1 GitHub Repository

All source code developed for this project — including the U-Net segmentation training notebook, the classification/ensemble training notebook, and the deployed Streamlit web application — is available at:

GitHub Repository: <https://github.com/SteveKevinDias/dermascanAI>

Repository contents:

- segmentation-skin.ipynb — U-Net segmentation model training
- classification-skin.ipynb — Multi-architecture classification training, TTA, ensembling, and Grad-CAM
- skinapp/ — Deployed Streamlit web application (app.py, utils/model.py, requirements.txt, README.md)

Note: Trained model checkpoints and the .env file are not included in the GitHub repository. Checkpoints are hosted separately on Hugging Face Hub (see Appendix D) and can be retrieved as described in the project README.

B.2 Kaggle Notebooks (Original Training Environment)

Model training was originally performed on Kaggle (GPU-accelerated, with direct access to the ISIC 2018/2019 dataset mirrors). The executed notebooks — including full training logs, output plots, and saved model artifacts — are available at:

Notebook	Purpose	Link
segmentation-skin.ipynb	U-Net lesion segmentation training	https://www.kaggle.com/code/stevedias01/segmentation-skin
classification-skin.ipynb	Multi-architecture classification, TTA,	https://www.kaggle.com/code/stevedias01/classification-skin

Notebook	Purpose	Link
	ensembling, Grad-CAM	

Note: The Kaggle notebooks reflect the exact environment and execution used to generate the results reported in Section 3 (Data Analysis and Results), including console outputs and training logs. The GitHub repository (C.1) contains the same code in a cleaned, version-controlled form for review and reuse outside Kaggle.

Appendix C — Additional Document Links

Document	Link
This project report (copy)	https://docs.google.com/document/d/1OhAr1FhfSfQ4iSb9EUVML8ofaIY6apox/edit
Trained model checkpoints (Hugging Face Hub)	https://huggingface.co/skd01/skin_cancer
Dataset — ISIC 2018 Task 1 (Kaggle mirror)	https://www.kaggle.com/datasets/trantoanthang/isic-2018
Dataset — ISIC 2019 Full (Kaggle mirror)	https://www.kaggle.com/datasets/alifshahariar/isic-2019-dataset-full